

中国科学院大学计算机组成原理实验报告

姓名	周禹驰	学号	2024K8009929027
专业	计算机科学与技术	实验编号	Prj4-选作 2
实验名称	功能型处理器结构优化:Cache 设计		

完整实验流程包括设计、开发、调试三个步骤：

- **设计**步骤为在代码编写前的准备阶段，主要包括 Cache 容量、组相联方式、地址划分、写策略、替换策略和旁路地址范围设计，并绘制状态机图纸。
- **开发**步骤为在 S-IDE 集成开发环境中进行 Verilog 代码编写，实现 I-cache、D-cache、tag/data array 访问和 CPU/内存侧握手。
- **调试**步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证，并通过 benchmark 运行时间和性能计数器观察 Cache 收益。

一、 设计成果(对设计流程的总结。需附上设计结构图,可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF,再插入到 L^AT_EX 中)

1 总体设计目标

本实验按照讲义要求,为 RISC-V32 流水线处理器设计独立的指令缓存和数据缓存。Cache 采用 Harvard 结构,I-cache 只处理取指读请求,D-cache 处理 Load/Store 请求。两者均采用 1024B 容量、4 路组相联、32B cache block 的结构;CPU 侧与内存侧接口均为 32bit,因此一次 cache line 填充或写回需要连续传输 8 个 word。

参数	I-cache	D-cache
容量	1024B	1024B
组织方式	4 路组相联	4 路组相联
Set 数	8	8
Block 大小	32B	32B
地址划分	tag[31:8], index[7:5], offset[4:0]	tag[31:8], index[7:5], offset[4:0]
写策略	只读	write-back, write-allocate

表 1: Cache 基本参数

2 组相联存储结构

每一路 Cache 包含 tag array 和 data array。tag array 保存 24 位 tag;data array 保存 256 位 cache block。valid、dirty 和替换指针由顶层寄存器数组维护。CPU 请求进入 Cache 后先锁存地址、写数据和写字节使能,后续状态机均使用锁存后的请求,避免 CPU 侧信号变化影响正在处理的 miss 或 write-back。

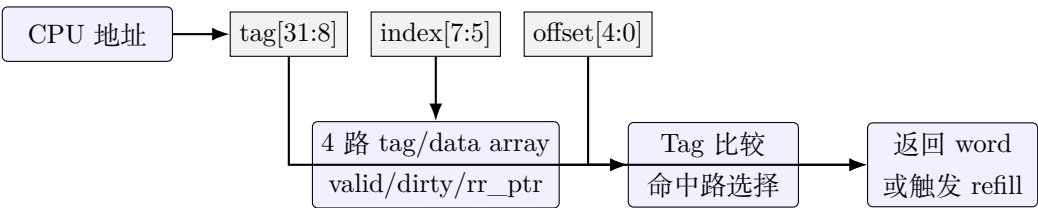


图 1: 4 路组相联 Cache 查询结构

3 I-cache 状态机设计

I-cache 为只读缓存。命中时从命中路读出对应 cache block,并根据 offset 返回目标指令;缺失时选择 victim 路,向真实内存发起 32B 对齐读请求,连续接收 8 个 word 后写入 data array 并更新 tag 和 valid。

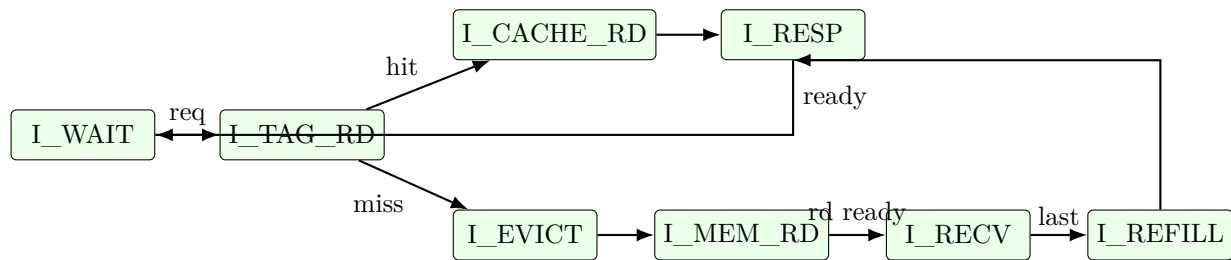


图 2: I-cache 状态机

4 D-cache 状态机设计

D-cache 支持读写请求,采用 write-back 和 write-allocate 策略。读命中直接返回目标 word;写命中通过 byte merge 函数根据 wstrb 修改目标 word 并置 dirty。读写缺失时先检查 victim 路,若 victim 有效且 dirty,则先写回旧 cache line,再读取新 cache line;若 victim 干净,则直接 refill。

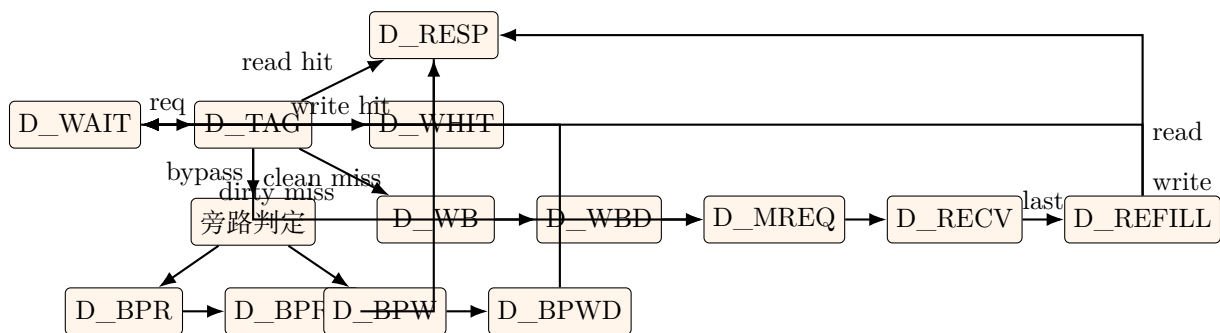


图 3: D-cache 状态机

二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 I-cache 关键代码

I-cache 的核心是命中判断、victim 选择和 refill buffer 组织。命中判断需要同时满足 valid 和 tag 相等;victim 选择优先无效路,否则使用轮转指针。refill 阶段将连续返回的 8 个 word 组成 256bit block,再写回 data array。

Listing 1: I-cache 命中判断逻辑摘要

```
assign hit_way0 = valid[set][0] & (tag_way0 == req_tag);
assign hit_way1 = valid[set][1] & (tag_way1 == req_tag);
assign cache_hit = hit_way0 | hit_way1 | hit_way2 | hit_way3;
```

Figure 4 展示了 I-cache 首次取指缺失后的完整 refill 过程。第 14 到 16 个周期,状态机停留在 I_MEM_RD 并向内存发出读请求;随后进入 I_RECV 等待返回数据。由于 AXI 侧 rvalid 不是连续拉高,8 个 word 分别 在第 22、23、26、27、28、35、37、39 个周期被接收,recv_cnt 随每次握手递增,最后一拍同时出现 rlast=1。

第 40 个周期状态机进入 `I_REFILL`, `fill_wen` 拉高, 将拼好的 32B cache line 写入选中的 cache way。第 41 个周期进入 `I_RESP`, `rsp_valid` 有效, 并从 refill buffer 中返回 PC 为 0x0 的指令 0x4137。之后 PC 为 0x4 的取指位于同一个 32B cache line 内, 状态机只经历 `I_TAG_RD`/`I_CACHE_RD`/`I_RESP`, 没有再次发起内存读请求, 说明 refill 后的 tag、valid 和 offset 选择均正确。

I-cache miss, 8-beat refill, then same-line hit

cycle	14--16	17--21	22	23	26	27	28	35	37	39	40	41	43--45
state	I_MEM_RD	I_RECV	I_RECV	I_RECV	I_RECV	I_RECV	I_RECV	I_RECV	I_RECV	I_RECV	I_REFILL	I_RESP	TAG/RD/RESP
rd_req	1	0	0	0	0	0	0	0	0	0	0	0	0
rvalid	0	0	1	1	1	1	1	1	1	1	0	0	0
rlast	0	0	0	0	0	0	0	0	0	1	0	0	0
recv_cnt	0	0	0	1	2	3	4	5	6	7	0	0	0
fill_wen	0	0	0	0	0	0	0	0	0	0	1	0	0
rsp_valid	0	0	0	0	0	0	0	0	0	0	0	1	1
inst	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x0	0x4137	PC+4 hit

PC=0x0 misses. Eight rvalid handshakes end with rlast at cycle 39; cycle 40 fills the line, cycle 41 returns from refill buffer.

The following PC=0x4 access stays in the same 32B line and reaches TAG/RD/RESP without another memory request.

图 4: I-cache 缺失、8 拍 refill 与随后命中返回

2 D-cache 关键代码

D-cache 写命中时不能简单覆盖整个 word, 而要根据 store 类型产生的 `wstrb` 进行字节合并。写缺失且采用 write-allocate 时, 也需要在 refill 的新 cache line 中合并本次写数据, 并将 dirty 置 1。

Listing 2: 按字节写使能合并写数据的逻辑摘要

```
function [31:0] merge_word;
    input [31:0] old_word;
    input [31:0] new_word;
    input [3:0] wstrb;
    begin
        merge_word[7:0] = wstrb[0] ? new_word[7:0] : old_word[7:0];
        merge_word[15:8] = wstrb[1] ? new_word[15:8] : old_word[15:8];
        merge_word[23:16] = wstrb[2] ? new_word[23:16] : old_word[23:16];
        merge_word[31:24] = wstrb[3] ? new_word[31:24] : old_word[31:24];
    end
endfunction
```

Figure 5 给出了 D-cache 写命中和写分配 refill 两个关键场景。第 587 个周期, CPU 侧发起对 0x3ff8 的 store 请求, `cpu_ready=1` 表示请求被接收; 第 588 个周期进入 `D_TAG` 后 `hit=1`, 第 589 个周期进入 `D_WHIT`, 通过 byte merge 写入 data array, 并维护 dirty 位。整个写命中路径没有向外部内存发起写事务, 符合 write-back 策略。

第 673 个周期,对 0x3fd8 的 store 请求在 D_TAG 阶段未命中;由于本次选中的 victim 为干净块,状态机没有进入写回状态,而是在第 674 到 676 个周期进入 D_MREQ 发起读请求。之后在 D_RECV 阶段接收 8 个 word,最后一拍出现在第 720 个周期并伴随 rlast=1;第 721 个周期进入 D_REFILL 写入新 cache line,第 722 个周期 cpu_ready 恢复为 1,等待中的 store 按 write-allocate 策略命中刚填入的 cache line。该波形说明写缺失时请求地址被锁存,refill 结束后能够正确响应 CPU 侧等待请求。

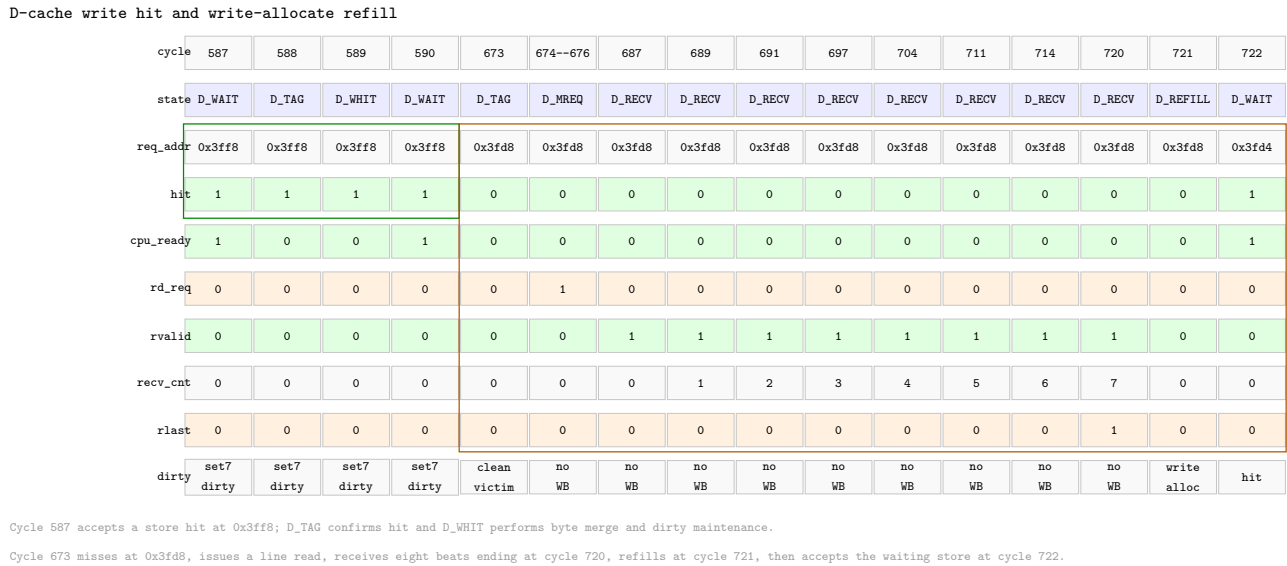


图 5: D-cache 写命中与写缺失 refill 后响应

需要说明的是,本次提供的 dump (1).fst 没有触发 dirty victim 替换:全局扫描 D-cache 状态机时,D_WB=4 和 D_WBD=5 在整个波形中均未出现。因此该 dump 不能作为“dirty miss 写回”截图依据,只能验证写命中置 dirty、clean miss 直接 refill 以及 refill 后响应。若后续补充触发脏块替换的仿真,预期波形应满足以下判据:D_TAG 阶段 hit=0 且 valid[idx][victim]&dirty[idx][victim]=1,随后进入 D_WB 发出 wr_req,在 D_WBD 连续发送 8 拍 wvalid/wready 数据并在最后一拍拉高 wlast,写回完成后再进入 D_MREQ/D_RECV/D_REFILL 读取新 cache line。

3 旁路地址处理

为保证 I/O 和 DNN 共享 DDR 区域的数据一致性,D-cache 对不可缓存区域采用旁路访问。当前旁路范围包括低地址特殊区域、I/O 空间以及 DNN 输入、权重、输出和参考结果所在区域。旁路读写均使用单 word 事务,避免外设寄存器或共享 DDR 数据被普通 cache line 缓存。

地址范围	处理方式	原因
addr < 0x20	旁路	低地址特殊区域
addr >= 0x4000_0000	旁路	MMIO 外设空间
0x0700_0000 <= addr < 0x0810_0000	旁路	DNN 共享 DDR 缓冲区

表 2: D-cache 旁路地址范围

4 与流水线 MEM 级的握手

流水线 MEM 级采用阻塞式访存,只有当 D-cache 接收请求并完成响应后,访存指令才继续进入 WB 级。该设计降低了 CPU 和 Cache 之间的乱序复杂度,能够保证 Load 写回和 Store 副作用的程序顺序。

5 无 Cache 基线性能分析

为了评估 Cache 带来的收益,先使用未加入 I-cache/D-cache 的流水线 CPU 作为基线,在 FPGA 上运行 coremark 和 dhrystone。两组日志均显示 FPGA 配置成功,benchmark 通过校验并最终进入 Hit good trap,因此该组数据可以作为功能正确的无 Cache 性能基线。

测试项	单次规模	五次 Total us	平均 Total us	输出结果
CoreMark	5 iterations	79,508,529;79,898,357;79,508,204;79,507,790;79,507,779	79,586,132	PASS
Dhrystone	100 runs	187,233;187,298;187,299;187,299;187,299	187,286	PASS

表 3: 无 Cache 流水线 CPU 的 FPGA 基线测试结果

从 Table 3 可以看到,两项测试的功能校验均通过,但耗时差异很大。CoreMark 平均一次运行约 79.59 s,5 次迭代对应的软件计算吞吐率只有约 0.063 iterations/s,因此日志中的 Iterations/Sec 由于整数除法被打印为 0。该现象不表示功能失败,而是说明无 Cache 时程序运行速度低到不足 1 iteration/s。

Dhrystone 平均一次运行约 187.29 ms,日志中的 Dhrystones per second 稳定在 533 到 534 之间。相比 CoreMark, Dhrystone 程序规模和工作集更小,控制流也更集中,因此虽然每条取指和访存仍然需要经过外部存储通路,整体运行时间仍明显短于 CoreMark。

无 Cache 基线暴露出的主要瓶颈是访存路径:取指请求、Load/Store 请求都直接进入外部存储系统,流水线在 IF 级和 MEM 级频繁等待 ready/valid 握手完成。CoreMark 包含链表、矩阵和状态机等访问模式,指令流和数据访问都更容易触发长时间等待,所以对 Cache 更敏感。加入 I-cache 后,循环体和热点函数的取指可以由片上 cache line 命中返回;加入 D-cache 后,栈、全局变量和局部数据结构中的重复访问可以减少 DDR 事务数量。因而该组无 Cache 数据主要用于后续与 Cache 版本比较:若 Cache 实现正确,性能提升应主要体现在 CoreMark 这类指令和数据局部性较强、外部访存开销占比高的程序上。

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

- **Cache 问题补充。**Cache 调试最容易忽略的是“地址已经改变但状态机还在处理旧请求”。因此 CPU 请求进入 Cache 后必须锁存地址和写数据,miss、write-back、refill 全过程都使用锁存值。

- **其他关键模块问题补充。**data_array.v 属于实验框架提供的存储阵列文件,不应为了规避拥塞或修复逻辑问题而修改;相关问题应优先在 Cache 控制状态机、旁路策略和外围时序上解决。

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

● 思考题 1:为什么 D-cache 需要对 MMIO 空间旁路?

MMIO 寄存器不是普通内存,读写通常具有副作用。例如读 done 寄存器需要真实访问外设状态,写 start 寄存器需要真实触发硬件任务。如果 MMIO 访问被 D-cache 缓存,CPU 可能反复读到旧值,或者写操作只停留在 cache 中而没有到达外设。

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	逻辑电路设计	D-cache	Store 后 Load 读到旧数据	写命中未正确按字节合并或 dirty 位未维护	使用 byte merge 并在写命中、写缺失后置 dirty
2	行为仿真	I-cache	miss 后返回指令错误	refill word 顺序或 offset 选择错误	按 offset[4:2] 选择 32bit word 并检查 8 拍接收顺序
3	系统一致性	DNN 共享区	DNN 测试超时或结果错误	共享 DDR 被 D-cache 缓存, CPU 和 DNN 看到的数据版本不同	对 DNN 数据区和 MMIO 区使用旁路访问

表 4: Cache 问题总结

类别	名称	体会	后续计划
数字电路设计	状态机划分	Cache miss 处理包含写回、读入、更新和响应多个阶段,必须清晰拆分	优化 miss 路径拍数,降低阻塞时间
RTL 编写	请求锁存	复杂外设握手中不能直接依赖 CPU 侧实时输入	形成固定的请求寄存器命名规范
验证方法	波形定位	Cache 错误通常要同时观察 CPU 侧请求和内存侧 burst 传输	建立 hit/miss/write-back/refill 四类波形样例

表 5: 收获总结

● 思考题 2: write-back 策略相比 write-through 有什么性能和正确性权衡?

write-back 可以把多次写命中合并到 cache line 中,减少内存写流量,性能通常更好;代价是必须维护 dirty 位,并在替换脏块时正确写回。对于与外设共享的数据区域,如果没有 cache flush 或旁路机制,write-back 还会带来一致性风险。

五、实验时长

在课后,你花费了大约 24 小时完成此次实验,其中各个阶段的用时如表 6 所示。

步骤	时长(小时)	说明
设计	3	Cache 参数、写策略、替换策略和旁路范围设计
开发	14	I-cache、D-cache 状态机和 CPU 握手实现
调试	6	hit/miss/refill/write-back 和 DNN 一致性验证

表 6: 实验时长